Journal of the Operational Research Society

ISSN: 0160-5682 (Print) 1476-9360 (Online) Journal homepage: https://orsociety.tandfonline.com/loi/tjor20

Solving the Greek school timetabling problem by a mixed integer programming model

Ioannis X. Tassopoulos, Christina A. Iliopoulou & Grigorios N. Beligiannis

To cite this article: Ioannis X. Tassopoulos, Christina A. Iliopoulou & Grigorios N. Beligiannis (2019): Solving the Greek school timetabling problem by a mixed integer programming model, Journal of the Operational Research Society, DOI: 10.1080/01605682.2018.1557022

To link to this article: https://doi.org/10.1080/01605682.2018.1557022

JOURNAL OF THE OPERATIONAL RESEARCH SOCIETY
https://doi.org/10.1080/01605682.2018.1557022

Solving the Greek school timetabling problem by a mixed integer programming model

Ioannis X. Tassopoulos[a], Christina A. Iliopoulou[b] and Grigorios N. Beligiannis[a]

[a]Department of Business Administration of Food and Agricultural Enterprises, University of Patras, Agrinio, Greece; [b]School of Rural and Surveying Engineering, National Technical University of Athens, Athens, Greece

ABSTRACT

This study deals with the school timetabling problem for the case of Greek high schools. At first, the problem is modelled as a Mixed Integer Programming problem for ten instances referring to Greek high schools. Then, the problem is coded using the MathProg programming language. Two different linear programming solvers are employed, Gurobi and CPLEX, to solve the problem for the instances at hand. Two methodologies are proposed. The first one deals with the problem utilising a model that includes all hard and soft constraints, called "monolithic" model, while the second one is based on a decomposition of the problem to six sub-problems. It should be stated that Gurobi and CPLEX did not produced satisfactory results when the monolithic model was the case. Computational results demonstrate the effectiveness of the second proposed methodology, as optimal solutions or new lower bounds were found. In addition, the results produced by Mixed Integer Programming are compared with the best so far published results, obtained by two Nature Inspired algorithms namely Particle Swarm Optimization and Cat Swarm Optimization.

ARTICLE HISTORY
Received 7 March 2018
Accepted 23 November 2018

KEYWORDS
School timetabling; mixed integer programming; linear programming solvers; mathprog programming language

1. Introduction

In the current work, we introduce two mixed integer programming (MIP) methods to solve the school timetabling problem of Greek high schools. The first one is a straight method while the second one is based on a decomposition of the problem to six other sub-problems. Our motivation was to find optimal solutions or, in case this would be impossible, to discover new lower bounds for a specific data set. This data set has been the focus of several research papers (Beligiannis, Moschopoulos, Kaperonis, & Likothanassis, 2008; Beligiannis, Moschopoulos, & Likothanassis, 2009; Katsaragakis, Tassopoulos, & Beligiannis, 2015; Raghavjee & Pillay, 2013, 2015; Skoullis, Tassopoulos, & Beligiannis, 2017; Tassopoulos & Beligiannis, 2012b; Zhang, Liu, M'Hallah, & Leung, 2010) and as a result, it constitutes a well-established test set instance in the respective literature (Kristiansen & Stidsen, 2013; Pillay, 2014). Although this data set was used by the above-mentioned researchers none of them has presented optimal values or dual bounds. Three instances taken from this data set were used at The Third International Timetabling Competition (ITC2011). This competition ran during 2011–2012 "with the aim of raising the profile of automated high school timetabling" as it is mentioned by Post, Gaspero, Kingston, McCollum, and Schaerf (2013). The participants had to deal with 35 instances from 10 countries. The authors of the present contribution coded the input data file of three instances into the XML format required by the ITC2011, specifically instances 3, 4, and 5, handing in them to the organisers who then used them as part of the benchmark data set. As far as instance 4 is concerned, none of the finalists achieved the lowest known cost value. It should be clear that the 10 instances we use at the current work are perhaps of medium size. Nevertheless, it is not easy to be solved by a soft computing algorithm. This is indicated by the fact that the algorithms proposed by Tassopoulos and Beligiannis (2012b) and by Skoullis et al. (2017) cannot achieve the lower bound or the optimal value the Integer Programming method presented in this contribution produces in all cases (see section 5), even though they constitute the-state-of-the art in the field. As it is stated in section 4.1 below, most of these instances constitute a very difficult data set for both two commercial solvers we used when a monolithic model of the problem is utilised. In our opinion, the former and the latter facts justify that although this data set is perhaps of a medium size it is not easy to be solved, hence it is not negligible.

The contribution of the present paper is three-fold. First, the school timetabling problem related to the aforementioned data set is modelled as a Mixed Integer Programming (MIP) problem and subsequently, the MIP model is coded using the GNU MathProg[1] programming language. The latter is an open source

CONTACT Ioannis X. Tassopoulos johnytass@gmail.com Department of Business Administration of Food and Agricultural Enterprises, University of Patras, G. Seferi 2, Agrinio 30100, Greece

programming language and a subset of the popular commercial language AMPL.[2] It should be noted that we employ a decomposition approach of the problem, since both commercial solvers, namely CPLEX and Gurobi, were unable to produce good quality solutions although they allowed several days to run. The decomposition approach we followed is explained in detail in section 4.1 and, to the best of our knowledge, is a novel one. Another novelty is the treatment of co-teaching of type I which is explained in section 3.2.4. Finally, the modellings of soft constraint S2 and the treatment of soft constraint S3 are novel ones in the IP relative domain. S2 soft constraint deals with the even distribution of each teacher's workload among the days of the week she/he is available at school while S3 soft constraint demands that a subject should not be taught more than once to any class during a day. The treatment of soft constraint S3 is inspired by the work of Tassopoulos and Beligiannis (2012a). The interested reader would realise these novelties by studying the subsections 3.3.2 and 3.3.3 respectively.

Secondly, we solve the problem for ten instances using the commercial MIP solvers Gurobi[3]

and CPLEX[4] proving optimal solutions or producing new lower bounds.

Thirdly, we compare the results produced by two Nature Inspired algorithms with those of linear programming solvers. The two Nature Inspired algorithms we have chosen, namely Particle Swarm Optimization (PSO) and Cat Swarm Optimization (CSO), have produced the best-published results so far for this data set and as mentioned above they constitute the state-of-the-art in the field (Skoullis et al., 2017; Tassopoulos & Beligiannis, 2012b respectively). It should be clear that the instances, which the present work refers to, pertain to Greek schools and certainly reflect real world situations.

The structure of this paper is as follows: in section 2, we present some efforts to solve the school timetabling problem using Integer Programming methods. In addition, we present some recent developments via soft computing and we mention the works of the four finalists of the Third Timetabling Competition, which were devoted to the school timetabling problem. The school timetabling problem and its computational complexity are described in section 3 while in section 4 the methodology used for the MIP formulation is presented. The two solution methods we follow are demonstrated in section 4.4. Comparative results are presented in section 5. Finally, conclusions and future work are discussed in section 6.

2. Related work

In the last decades, several researchers have dealt with the school timetabling problem in a variety of solution approaches. The interested reader could study an excellent review of the attempts for solving the school timetabling problem made up to 1999, provided by Schaerf (1999). Moreover, the survey provided by Pillay (2014) contains interesting information about more recent efforts on this problem made up to 2014. Next, we refer to some efforts constituting MIP approaches and soft computing techniques, as well. Some of them are very recent, that is, published between 2014 and 2018.

Early literature on MIP models for school timetabling focussed on the specific problem faced by the secondary educational system in Greece. Birbas, Daskalaki, and Housos (1997) presented an integer programming approach for the timetabling problem of Greek high schools. The model aimed to schedule a large number of classes, teachers, courses, and classrooms to a number of time-periods. A column generation approach for this problem was developed in a later study by Papoutsis, Valouxis, and Housos (2003). Further, Valouxis and Housos (2003) presented a methodology, which consists of a combination of Constraint Programming and Operational Research techniques in order to solve the problem at hand for Greek high schools. In particular, they mainly used the Constraint Programming platform of CPLEX solver library, while by solving relaxed models using minimum cost matching algorithms they calculated problem lower bounds. These lower bounds are then used to prioritise the search process of the Constraint Programming procedure. A two-step methodology applied to Greek datasets was presented in the study by Birbas, Daskalaki, and Housos (2009), treating the shift assignment problem for teachers first and constructing the school timetable afterwards.

In recent years, advances in the computational efficiency of general-purpose MIP solvers have motivated researchers to investigate the potential of exact algorithms for the school timetabling problem (Bixby, 2012; Kristiansen, Sørensen, & Stidsen, 2015). Santos, Uchoa, Ochi, and Maculan (2012) presented a new integer programming formulation for a variant of the high school timetabling problem, namely the Class-Teacher Timetabling problem with Compactness requirements (CCTPCR). The authors used a column generation approach to generate strong lower bounds for a set of datasets from Brazil. Sørensen and Stidsen (2013) devised two new MIP models for high school timetabling in Denmark, providing comprehensive computational results for 100 real-life instances. Dorneles, de Araújo, and Buriol (2014) proposed a MIP model and a fix-and-optimise heuristic combined with variable neighbourhood descent search for the CCTPCR. Three different types of decomposition

(class, day, and teacher) were applied to handle the problem complexity, while new best solutions were found for seven instances. A special mention should be made to the work by Kristiansen et al. (2015). They formulated a MIP model solved in two phases by incorporating the soft constraints of the problem once the optimal value for the model containing the hard constraints was obtained. A commercial MIP solver was used, yielding optimal results and new lower bounds for real-life instances. The special mention is due to the fact that this work addresses all the instances of the Third International Timetabling Competition (ITC2011) specifically devoted to school timetabling. Since we contributed with three of our own instances to ITC2011, as we explained in section 1. Introduction, the proposed formulation is applicable to our instances, too. As a matter of fact, to the best of our knowledge, this is the only published IP formulation that is applicable to our benchmark data set, besides the one mentioned in the current work. For the sake of completeness we have to state that the main difference between the work by Kristiansen et al. and ours is that they have a more general scope, since their formulation is applicable to all of ITC2011 instances. In addition, their method is based on giving a solution considering the hard constraints first and the soft afterwards, while we present a solution concerning all hard and soft constraints, except the one referring to idle teacher hours, and afterwards we consider the excluded constraint separately for each day. Another difference is the IP formulation of the constraint referring to teacher idle times we use, which is borrowed from the work of Dorneles, de Araújo, and Buriol (2012). We think that this formulation is very sophisticated and performs better than the one presented by Kristiansen et al., which is a simple one. Besides, the former statement is justified by the analysis of the possible formulations of this constraint presented in the work by Dorneles et al. We refer the interested reader to section 4.2.2 and at the corresponding paper itself for more information. Another difference between Kristiansen et al.'s work and ours is the way we treat the co-teaching cases. We introduce the notion of "virtual teacher" by which the handling of these cases is simplified, as we believe. For more information we refer the interested reader to section 3.2.4. Finally, with respect to the relative performance of these two IP formulations, the one by Kristiansen et al. and ours, we could state that our formulation performs better. We make an extended justification comment on this in sub section 4.1. We next proceed with some other relatively recent efforts on solving the school timetabling problem. Al-Yakoob and Sherali (2015) proposed two decomposition approaches and a column generation solution framework, implemented in CPLEX-MIP, to handle instances of realistic size for high schools in Kuwait. Lately, Dorneles, de Araújo, and Buriol (2017) proposed a novel formulation based on a multi-commodity flow model for the CCTPCR, solved by a column generation method. Experimental results showed that the method produced high quality solutions, achieving new lower bounds for five instances from the literature in short computational times.

Besides the aforementioned works that are based on Integer Programming there are a lot of soft computing algorithms and heuristics that have been applied to the school timetabling problem producing promising results. We just mention some of the most recent ones. Saviniec, Santos, and Costa (2018) present some variants of an algorithm based on parallelism. They compare the performance of different parallel algorithm variants against some stand-alone metaheuristics, such as iterated local search and adapt versions of tabu search, simulated annealing, and late acceptance strategy, and against two state-of-the-art algorithms by Dorneles et al. (2014) and Fonseca, Santos, and Carrano (2016). They finally conclude that their algorithm outperforms all other algorithms including the two state-of-the-art ones. Saviniec and Constantino (2017) present a soft computing algorithm based on Iterated Local Search and Variable Neighborhood Search metaheuristic frameworks. They use seven public instances and a new dataset with 34 real-world instances including large cases to validate their algorithm. Their algorithm manages to outperform existing approaches and achieves the best solution in 38 out of 41 instances. We complete the short reference to the recent soft computing achievements by citing the efforts of the finalists of the Third International Timetabling Competition (ITC2011), specifically devoted to school timetabling, where state-of-the art timetabling solvers were extensively evaluated. The finalists of the competition were actually four teams. Fonseca, Santos, Toffolo, Brito, and Souza (2012) present an approach based on Simulated Annealing (SA) and Iterated Local Search (ILS). First, an initial solution is generated using KHE School timetabling engine created by Kingston (Kingston 2012) and then SA and ILS are utilized to refine the quality of this solution. Domrös and Homberger (2012) present an Evolutionary Algorithm. Their solver is based on two ideas: First, each coded solution is represented by a permutation of sub-events. Second, the evolutionary search is driven by the population concept of $(1+1)$-Evolution Strategy. This solver is able to produce solutions of the instances of ITC2011. However, in round 1 of the competition the solutions are not the best known or the optimal ones. Kheiri, Özcan, and Parkes (2012)

propose an approach where a developed hyper-heuristic intelligently controls the selection of a single move operator among a range of neighborhood operators. Kheiri et al. suggest that their approach can easily be adopted to solve some related problems such as university course timetabling and exam timetabling, too. This is due to the abstraction of their approach. Finally, Sørensen, Kristiansen, and Stidsen (2012) propose an approach that is based on Adaptive Large Neighbourhood Search (ALNS). Their framework consists of three different strategies, namely remove strategy, adapt strategy and accept strategy. These strategies are responsible for the insertions and the removes applied on the current solution at each iteration and for selecting the candidate solution. Since this algorithm has a very good performance on school timetabling problem Sørensen et al. propose the use of it to other similar problems, too.

3. The school timetabling problem and its computational complexity

Kristiansen and Stidsen (2013) report that the problem of timetabling refers to the assignment of available resources to objects placed in time in such a manner to satisfy as many desirable objective targets under the relevant constraints. This specific problem belongs to the broader class of timetabling problems, which also includes University course timetabling and exam timetabling. According to Pillay (2014), the school timetabling problem is NP-complete, depending on the constraints involved. Besides, if the constraint of teacher availability is taken into account, the problem becomes NP–complete, as proven by Even, Itai, and Shamir (1975). The former is a common constraint in school timetabling problems depicting actual conditions. In reality, this means that it is probably impossible to find a polynomial time algorithm that can solve the problem, while the solution time most probably rises at least exponentially in relation to the problem size. The interested reader is referred to literature where a more recent study of the problem's complexity is presented (Carter & Tovey, 1992; Cooper & Kingston, 1996; ten Eikelder & Willemen, 2001; Willemen, 2002).

3.1. The problem entities

The specific school timetabling problem (ie the Greek school timetabling problem) includes the following entities:

- **Period:** a structural block of time used in the design process of timetabling.
- **Day:** a day of the week, which consists of seven periods at most.

In pucrs are 8 periods

Maybe this can be replace for the idea of couse

- **Planning time:** the time period that the timetable refers to. It consists of five days.
- **Class:** a set of students who are timetabled together.
- **Teacher:** the class instructor for a predefined number of periods.
- **Subject:** the teaching object in a period.

3.2. School timetabling problem constraints

There are several published papers in the global literature dealing with the school timetabling problem (Pillay, 2014). However, there is no uniformly accepted unique definition of the problem's constraints, as these vary depending on the country involved in each case (Kristiansen & Stidsen, 2013). In the following sections, the soft and hard constraints of the nine instances we use are presented. These nine instances have been widely used in the respective bibliography as we have already mentioned in section 1.

3.2.1. Hard and soft constraints definition

Those constraints, that cannot be violated in any case, are called hard. The satisfaction of all hard constraints in a resulting schedule is what renders the latter feasible. In reality, this means that the schedule may actually be used by the educational organisation in question. The existence of hard constraints is accompanied by that of soft constraints. It is desired to satisfy these constraints, although it is not required. The satisfaction of them is directly linked to the quality of the schedule. The more soft constraints the schedule satisfies, the more efficient it is considered. It should be also noted that soft constraints constitute competing targets and thus, the satisfaction of the full set of soft constraints is typically impossible. The objective of solving the school timetabling problem for a specific dataset, using a certain MIP solver, is the development of a schedule that meets all hard constraints, and simultaneously satisfies the largest possible number of soft constraints. Next, we introduce the hard and soft constraints of the Greek high school problem at hand.

updating this is possible to add the ideia of making a teacher unavailable for AB, if he was schedule for NP in the day before.

The following hard constraints pertain to the instances in question:

H1. Each teacher, during any given period in a day that she/he is available, can teach at most one class, while she/he cannot teach any class during a day when she/he is unavailable.

H2. Each teacher can teach a class out of a specified number of classes for a predefined number of periods and only on days she/he is available to the school.

updating this is possible to add the ideia of workload per day.

H3. Each class, in any weekday, is associated with a maximum number of teaching hours.

H4. Each class, for a given period, is assigned to one teacher at most, except for the case of co-teaching.

H5. Each class has a compact schedule during any given day, while idle hours are allowed only at the end of each day.

H6. In some cases, two teachers must be assigned to the same period at the same class, for a specific number of periods. This is a type of co-teaching.

H7. Specific teachers must concurrently teach some classes for a predefined number of periods. This type of co-teaching differs from that referred to **H6** hard constraint.

3.2.3. Soft constraints' summary

The soft constraints, the satisfaction of which is desirable, are the following:

S1. Each teacher's daily schedule must be compact, meaning that there should be no idle hour between two teaching hours.

S2. Each teacher's weekly schedule should be evenly filled during the days she/he is available to the school.

S3. A subject should not be taught more than once to any class during a day.

3.2.4. The two types of co-teaching and their treatment

Constraints **H6** and **H7** pertain to cases referred to as co-teachings. Each one of these constraints refers to different cases of co-teaching. The first case, expressed in **H6**, is called "type I co-teaching", while **H7** refers to "type II co-teaching". It should be noted that instances 1, 2, 4 and 7 include type I co-teaching, while instances 2 and 4 additionally include type II co-teaching. The remaining instances do not include co-teaching cases. A full description of the input files is referenced by Tassopoulos and Beligiannis (2012a).

In order to treat type I co-teaching, ie model constraint H6 for each pair of teachers who must simultaneously teach a class for a specific number of periods and a number of subjects, we define a virtual teacher. This virtual teacher is been assigned the shared teaching hours of the class as well as the number of subjects for each class. At the same time, these hours are been subtracted from the schedules of the original pair of teachers. To illustrate, let us assume that based on the input data, teachers with code number 1 and 5 co-teach one subject to class 1 for 3 hours and one subject to class 7 for 4 hours. In addition, teacher 1 must teach for a total of 18 hours during the week, while teacher 5 has a total of 15 teaching hours. According to the proposed method, we define an additional virtual teacher who teaches one subject to class 1 for 3 hours and one to class 7 for 4 hours. Therefore, the total teaching hours of the virtual teacher are 7 a week. At the same time, teacher 1 is not assigned to teach any subject to neither class 1 nor class 7 in both cases, since her/his teaching hours are assigned to the virtual teacher. The same applies in the case of teacher 5. Now, teacher 1 is assigned 18 – 3 – 4 = 11 hours in total each week, while teacher 5 is assigned 15 – 3 – 4 = 8 hours a week. It must be noted that virtual teachers are not required to comply with constraint S1 pertaining to idle hours in their schedule. Such a thing would be meaningless, as we are only concerned about idle hours in the case of actual teachers, once their teaching hours are calculated using virtual teachers. For instance, let us assume that teacher 1 must teach by himself during the 1st, 2nd, and 4th hour on Monday. In addition, let us assume that the virtual teacher consisting of teachers 1 and 5 has a class on Monday during the 3rd and 5th hour. Then teacher 1 in reality has to teach for 5 consecutive hours (1st and 2nd by himself, 3rd with teacher 5, 4th by himself and 5th with teacher 5). Thus, obviously there is no idle hour in his schedule on Monday. As we state in section 1, the former proposed approach of dealing with co-teachings of type I is, to the best of our knowledge, a novel one.

Regarding constraint **S2** (that we call it "teacher dispersion"), we must remind the reader that the daily total of teaching hours for each teacher t must lay within the interval $[LowerBound[t],\ UpperBound[t]]$, where:

$$LowerBound[t] = floor\left(\frac{totalTeachingHours[t]}{AvailableDaysNumber[t]}\right) \tag{1}$$

and

$$UpperBound[t] = ceiling\left(\frac{totalTeachingHours[t]}{AvailableDaysNumber[t]}\right) \tag{2}$$

with $totalTeachingHours[t]$ representing the total teaching hours for teacher t on a weekly basis and $AvailableDaysNumber[t]$ representing the number of days during which teacher t is available. Constraint **S2** does not obviously pertain to virtual teachers. If a teacher is not a virtual one and is not teaching a co-teaching -thus she/he is not considered a part of

a virtual teacher- then constraint **S2** directly applies in her/his case. In the case, however, where a teacher is considered as a part of one or more virtual teachers, then in Equations (1) and (2), the variable $totalTeachingHours[t]$ does not represent teaching hours for teacher t exclusively, but also takes into account the hours of all the virtual teachers she/he is considered a part of. The values of $LowerBound[t]$ and $UpperBound[t]$ are calculated for each teacher t that is not a virtual one and are given as input in the model that we will subsequently present.

Moreover, the availability days in the case of a virtual teacher are the intersection of the sets of all availability days of the ordinary teachers she/he consists of. Finally, constraint **S3** applies to both the cases of ordinary and virtual teachers.

Despite the fact that treating type I of co-teaching requires sophisticated operations and restructure of the input data, the case of type II of co-teaching is relatively simple. Therefore, in order to model constraint **H7**, it suffices to include an additional constraint for each case of this type of constraint, which will ensure that when one teacher teaches one of the two classes, the other will teach the other class.

3.3. Soft constraint violation cost

Under the requirement that all hard constraints are satisfied, we present the calculation process for soft constraints cost.

3.3.1. Evaluation of constraint S1 (teacher idle periods)

For every idle hour in the schedule of any teacher (which is not a virtual one), a cost unit is attributed (violation of constraint **S1**).

3.3.2. Evaluation of constraint S2 (teacher dispersion)

This soft constraint deals with the even workload distribution of each teacher among the days of the week she/he is available at school. To the best of our knowledge, in previous published works where several versions of this constraint are present, the bounds of the workload of each teacher for each day are fixed and given as an input. If this is the case then the task of modelling the corresponding soft constraint is easy to accomplish. It suffices to aggregate the teaching ours of a teacher for a day and demand that this summated amount of teaching hours is between the lower and upper bounds. This of course should be done for any day and for any teacher. Since the constraint that it is considered at the present work is slightly different and somehow more flexible or general, the above-mentioned approach is inapplicable. Therefore, we propose a novel evaluation method for the violation cost of constraint **S2**. For the case of every teacher where the actual dispersion of teaching hours does not match the ideal dispersion, one cost unit is attributed. For instance, assuming that a teacher has a total of 19 hours per week and is available for 5 days, her/his ideal dispersion is given by the sequence: 4, 4, 4, 4, 3 and by any permutation of these numbers. However, in the case that the actual dispersion is the following: 5, 3, 3, 5, 3, then the cost is not equal to 2, as someone observing that the teaching hours on Monday and Thursday are $5 > 4$ (actual > ideal) could hastily assume. In reality, the cost is equal to 4, as on both Monday and Thursday the Upper Bound is exceeded by one hour (so 2 cost units are attributed), while the number of times the Lower Bound (ie 3) appears in the actual dispersion is 3 (Tuesday, Wednesday and Friday), when it should be 1 (on any day). Consequently, 2 more cost units added to the total cost of the specific actual dispersion, which becomes 4. In the next sections, we present the equation for calculating the violation cost for constraint **S2**, in accordance to the above-explained reasoning.

Now, we introduce some fundamental parameters and decision binary variables that are utilised to calculate the S2 soft constraint violation cost. First, let $X[t,\ c,\ d,\ p]$ be a binary variable, which takes the value of 1 if teacher t teaches class c on day d and period p and 0 otherwise. As it is obvious, the sum $\sum_c \sum_p X[t,c,d,p]$ represents the teaching hours for teacher t on day d. This sum has been used at the definition of $D_1[t,d]$ and $D_2[t,d]$ binary decision variables that follows. Moreover, let $TimesLowerBound[t]$ denote the appearance frequency of the $LowerBound[t]$ in the case of the ideal dispersion and $TimesUpperBound[t]$ denote the appearance frequency of the $UpperBound[t]$ in the case of the ideal dispersion. Since the ideal distribution for each teacher is known in advance, the latter parameters take their values directly from the input data. Moreover, these two parameters are also used for defining the two decision variables, namely $D_1[t,d]$ and $D_2[t,d]$, the definition of which is given in the next lines. Finally, we introduce two more decision variables. The first is $DaysEqualLowerBound[t]$, which denotes the number of days on which teacher t has a total of teaching hours equal to $LowerBound[t]$. The second is $DaysEqualUpperBound[t]$, which denotes the number of days on which teacher t has a total of teaching hours equal to $UpperBound[t]$. The latter variables are needed for the definition of the binary variables $D_3[t]$ and $D_4[t]$. Below, we present the definition of the aforementioned binary decision variables for each teacher $t \in T$ and day $d \in D$:

$$D_1[t,d] = \begin{cases} 1, & if \ \sum_c \sum_p X[t,c,d,p] < LowerBound[t] \\ 0, & otherwise \end{cases} \tag{3}$$

$$D_2[t,d] = \begin{cases} 1, & if \ \sum_c \sum_p X[t,c,d,p] > UpperBound[t] \\ 0, & otherwise \end{cases} \quad (4)$$

$$D_3[t,\ d] = \begin{cases} 1, & if \ DaysEqualLowerBound[t] > TimesLowerBound[t] \\ 0, & otherwise \end{cases} \quad (5)$$

$$D_4[t,\ d] = \begin{cases} 1, & if \ DaysEqualUpperBound[t] > TimesUpperBound[t] \\ 0, & otherwise \end{cases} \quad (6)$$

Binary variable $D_1[t,d]$ is set to 1 if and only if the teaching hours of a teacher during a day are less than they should be, while binary variable $D_2[t,d]$ is set to one if and only if the teaching hours of a teacher during a day are more than they should be. In other words, the binary variables $D_1[t,d]$, $D_2[t,d]$ partially signify the violation of the ideal teaching hour distribution for a teacher t during a day d. The fact that $D_1[t,d] = 0$ and $D_2[t,d] = 0$, does not ensure that the cost of constraint S_2 for a teacher is zero. The S_2 cost for a teacher is zero if in addition to the latter condition, $D_3[t] = 0$ and $D_4[t] = 0$. In order to clarify this point, please refer to the example of the first paragraph of section 3.3.2.

Now, the violation cost for constraint **S2**, is given by the formula:

$$S_2 \ \cos t = \sum_t \sum_d D_1[t,d] + \sum_t \sum_d D_2[t,d] + \sum_t D_3[t] + \sum_t D_4[t] \quad (7)$$

More details about the S2 MIP formulation are given in section 4.2.2.

3.3.3. Evaluation of constraint S3 (class dispersion)

For every day during which a subject is taught for more than one hour in a class, one cost unit is attributed (violation of constraint **S3**). It should be noted that, for the sake of reducing modelling complexity, we do not take into account the type of subject taught by any teacher. What is, finally, of importance to us is the number of subjects taught by each teacher to each class he is assigned to. Under this simplification, we are able to check for violations of constraint **S3.** This is because if a teacher teaches, for instance, two subjects and is assigned two teaching hours on that day in a class, then it is certain that no violation occurs. This happens because it is possible to teach the first subject during the first hour and the second one during the following hour. On the contrary, if a teacher is assigned three hours on a particular day in a class, but must teach two subjects, then it is certain that one of the two must be taught for a second hour. Therefore, the criterion for violation of constraint **S3** is the number of teaching hours in a day in relation to the number of subjects taught by the teacher on that day for a specific class. If the number of teaching hours exceeds the number of subjects taught by the teacher to the class, then a violation of constraint **S3** occurs. It should be noted that the number of subjects to be taught by a teacher to a class is given as input. Finally, we must observe that the treatment we propose is only applicable when there is no case where the difference between any two subjects in terms of total teaching hours is greater than one hour. Please note that the problem of Greek high school timetabling at hand satisfies the former condition.

4. Methodology

4.1. IP formulations and our benchmark data set

In the following model, the soft constraints have been incorporated in the objective function, the value of which we aim to minimise under the hard constraints. At this point, we must report that our attempt to solve the problem in an acceptable computational time by simultaneously incorporating all three soft constraints in one model was not successful. None of the solvers CPLEX and Gurobi were able to find a good quality solution, even though the time limit for algorithm termination was set to several days. The only exceptions noted were in the case of instances 5 and 8, which are however, considered as very easy to handle instances. At this point we should refer to the work of Kristiansen et al. (2015) where an IP formulation capable of handling all of the ITC 2011 instances is presented. As a consequence, this IP formulation is applicable to our benchmark set, too. Nevertheless, there is strong evidence that the formulation proposed by Kristiansen et al. (2015) is not better than ours. We present a part from the abstract of that paper that praises the efficacy of that formulation: "*Computational results show that our approach is able to find previously unknown optimal solutions for 2 instances of XHSTT, and proves optimality of 4 known solutions. For the instances not solved to optimality, new non-trivial lower bounds were found in 11 cases, and new best-known solutions were found in 9 cases. Furthermore, the approach is shown to be competitive with the finalist of Round 2 of the International Timetabling Competition 2011.*" Despite these achievements, regarding the three instances we have provided to the competition organisers (see section 1. Introduction) we have to underline the following facts: first, the three instances, namely 3, 4 and 5 were not the most difficult ones among those possessed by us. Observing Table 3(a) would reveal that instance 3 has an order of difficulty equal to 6, while instance 4 has an order of 5. Finally, instance 5 has an order of difficulty equal to 10, ie it is the easiest to be solved. Second, the

scores yielded by that formulation were 25, 81 and 15 for instances 3, 4 and 5, respectively (Kristiansen et al., 2015). However, even our monolithic model performs better than this, since as Table 4(b) reveals, the corresponding costs are 7, 15 and 0. We think that assuming worse performance of Kristiansen et al.' s formulation compared to our model for the rest, more difficult, instances would be rather justified. Another safe assumption would be that, since the formulation of Kristiansen et al. has such a success in ITC 2011 instances but at the same time it fails so badly for the easy to solve instances 3, 4, and 5, then the benchmark instances we use have some noticeable difficulty to solve, at least some of them. Here, we have to admit that, to the best of our knowledge, there is no other published IP formulation that is applicable to our benchmark data set despite the fact that virtually an infinite number of possible Integer Programming Formulations exist. In order to reduce the difficulty of the problem at hand, we adopted a different approach of decomposing the initial problem to six sub-problems. We explain this 2nd decomposition approach in section 4.4. In addition, in this section we present some information about the difficulty of the initial problem by giving the number of constraints, variables and non-zeroes of its model.

4.2. MIP formulation of the constraints

Although the formulation of hard constraints does not present any particular difficulty we present it below, in subsection 4.2.1, to ensure the readability and completeness of the text. On the contrary, the formulation of the soft constraints requires special attention and presents an interesting case. In the following subsections 4.2.2, 4.2.3, and 4.2.4, we demonstrate explanatory comments for the formulation of each soft constraint, while the exact coding in MathProg presenting hard and soft constraints is available online at the University of Patras web site.[5]

4.2.1. Hard constraints MIP formulation

The hard constraints of the dealt problem are seven, namely H1, H2, H3, …, H7 and have been already presented in subsection 3.2.2 above. At this point we introduce their algebraic form.

First, we define the binary variables:

$$X[t,\ c,\ d,\ p] = \begin{cases} 1, & \textit{if teacher } t \textit{ has a lesson at class } c \\ & \textit{at day } d \textit{ and period } p \\ 0, & \textit{otherwise} \end{cases} \tag{10}$$

H1.

$$\sum_{c} X[t,\ c,\ d,\ p] \leq\ 1$$

for any teacher t, any class c, any day d with $d = 1, 2, \ldots, 5$ and $d = d^*$TeacherUnavDays$[t,\ d]$, and any period p of the day d. Please note that TeacherUnavDays$[t,\ d]$ is a two dimensional matrix with values 0 and/or 1. The value 0 at the cell $[t,\ d]$ of the matrix indicates that teacher t is unavailable on day d, while the value 1 means that the teacher t is available on day d. The values of this matrix are given as an input. Moreover, the equality $d = d^*$TeacherUnavDays$[t,\ d]$ ensures that for each teacher only the days she/he is available at school are considered.

H2 and H3.

$$\sum_{d,\ p} X[t,\ c,\ d,\ p] = \textit{TeacherClassHours}[t,\ c]$$

for any teacher t and any class c. The summation is done over the days d with $d = d^*$TeacherUnavDays$[t,\ d]$, and over the periods p of the day d. Please note that TeacherClassHours$[t,\ c]$ is a two dimensional matrix that is given as input. Any cell $[t,\ c]$ of this matrix contains the predefined number of teaching hours of teacher t in class c.

H4.

$$\sum_{t} X[t,\ \ c,\ d,\ p] \leq\ 1$$

for any class, for any d with $d = d^*$TeacherUnavDays$[t,\ d]$, and any period p of the day d.

H5.

$$\sum_{t} X[t,\ c,\ d,\ 1] = 1\ \ (a)$$

for any teacher t and any day d.

$$\sum_{t} X[t,\ c,\ d,\ p{-}1] - \sum_{t} X[t,\ c,\ d,\ p] \geq 0\ \ (b)$$

for any teacher t, any day d and any period p of the day d with $p = 1, 2, \ldots, 5$.

The constraints (a) ensure that any class will assigned a teaching at the first time of each day of the week, while the constraints (b) ensure the compactness of the daily schedule of each class. Additionally, constraints (b) allow any possible idle period of a class occurring only at the end of the day.

H6. The modelling of hard constraint H6 which refers to co-teaching of type I (see sub section 3.2.4) is already covered by the modelling of H4 by considering the virtual teachers as ordinary ones (see sub section 3.2.4). However, we have to ensure that when a virtual teacher t, composed by the ordinary teachers $t1$ and $t2$, teaches at a period p then none of the $t1$ and $t2$ is teaching at this period and vice

versa. This constraint is modelled as:

$$\sum_c X[t,\ c,\ d,\ p] + \sum_c X[t1,\ c,\ d,\ p] \leq 1$$

and

$$\sum_c X[t,\ c,\ d,\ p] + \sum_c X[t2,\ c,\ d,\ p] \leq 1$$

for any day d, any period p of d and any pair of teachers $t1$ and $t2$ which are members of a virtual teacher t.

H7. If the teachers $t1$ and $t2$ must teach at the same period in different classes, say $c1$ and $c2$, then the corresponding constraint is:

$$X[t1,\ c1,\ d,\ p] = X[t2,\ c2,\ d,\ p]$$

for any day d and any period p of the day d.

4.2.2. S1 MIP formulation

Regarding soft constraint **S1**, we have adopted the 4th model proposed by Dorneles et al. (2012). In this specific paper, four alternative modelling approaches for constraint **S1** are presented, with the last one considered as the most efficient. For the sake of brevity, we will refrain from describing the method in detail and will instead refer the interested reader to the paper itself.

4.2.3. S2 MIP formulation

We will present the formulation for soft constraint **S2**, according to what we explained in section 3.3.2. In addition, we use the *Big M* method. The latter method is employed in order to express conditional constraints as linear ones.

Let us define the next integer variable for any non-virtual teacher t:

$$U[t,\ d] = LowerBound[t] - \sum_c \sum_p X[t,c,d,p]\ \ \forall t \in T, d \in D \tag{11}$$

Based on instance 2 and on the formula given by equation (1) in section 3.2.4, we have computed parameters $LowerBound[t]$ for every non-virtual teacher t. The values of these parameters are given in the data file accompanying the code file. This data file, among others, is available online.[6]

In the case of data file 2, that we utilise as an example, we can observe the following:

$$1 \leq LowerBound[t] \leq 3\ \ \forall t \in T \tag{12}$$

$$0 \leq \sum_c \sum_p X[t,c,d,p] \leq 7\ \ \forall t \in T,\ \ d \in D \tag{13}$$

Based on equation (11), and inequalities (12) and (13) the following is true:

$$-MU2 \leq U[t,d] \leq MU1\ \ \forall t \in T,\ \ d \in D \quad where\ \ MU1 = 4,\ \ MU2 = 6 \tag{14}$$

Next, for each $t \in T$ and $d \in D$ we define the binary variable:

$$D_1[t,d] = \begin{cases} 1,\ \ if\ \ U[t,\ \ d] > 0 \iff \\ \sum_c \sum_p X[t,c,d,p] < LowerBound[t] \\ 0,\ \ otherwise \end{cases} \tag{15}$$

Let us also define for each $t \in T$ and $d \in D$ the following integer variable:

$$UU[t,\ \ d] = \sum_c \sum_p X[t,c,d,p] - LowerBound[t] \tag{16}$$

As before, it can be easily proven that:

$$-MU1 \leq UU[t,d] \leq MU2\ \ \forall t \in T,\ \ d \in D \quad where\ \ MU1 = 4,\ \ MU2 = 6 \tag{17}$$

We now define for each $t \in T$ and $d \in D$ the binary variable:

$$D_{11}[t,d] = \begin{cases} 1,\ \ if\ \ UU[t,\ \ d] > 0 \iff \\ \sum_c \sum_p X[t,c,d,p] > LowerBound[t] \\ \iff \sum_c \sum_p X[t,c,d,p] \geq UpperBound[t] \\ 0,\ \ otherwise \end{cases} \tag{18}$$

The equivalence between the two different conditions in the definition of variable $D_{11}[t,d]$ is because $UpperBound[t] = LowerBound[t] + 1$.

It is true that:

$$K[t,\ \ d] = \sum_c \sum_p X[t,c,d,p] - UpperBound[t] \quad \forall t \in T, d \in D \tag{19}$$

Since $1 \leq UpperBound[t] \leq 4$, as in instance 2, the following is true:

$$-MK2 \leq K[t,\ \ d] \leq MK1\ \ \forall t \in T,\ \ d \in D \quad where\ \ \mathrm{MK1} = 6,\ \ \mathrm{MK2} = 4 \tag{20}$$

Also, let us define for each $t \in T$ and $d \in D$ the following variable:

$$D_2[t,\ \ d] = \begin{cases} 1,\ \ if\ \ K[t,\ \ d] > 0 \\ \iff \sum_c \sum_p X[t,c,d,p] > UpperBound[t] \\ 0,\ \ otherwise \end{cases} \tag{21}$$

In addition, we introduce parameters $Teacher\ UnavDays[t,\ \ d]$, the values of which are given in the input file and are either 1 or 0. The value of 1 corresponds to the case of teacher t being available on day d, while 0 to the opposite case.

Moreover, we compute the following parameters:

$$AvailableDaysNumber[t] = \sum_{d} TeacherUnavDays[t,d] \ \forall t \in T \quad (22)$$

which represent the number of days during which each teacher t is available to the school.

Finally, we define the following auxiliary variables:

- $0 \leq$ ***DaysGreaterEqualUpperBound*[t]** $\leq$ *AvailableDaysNumber*[t] which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours greater or equal to the *UpperBound*[t]
- $0 \leq$ ***DaysGreaterUpperBound*[t]** $\leq$ *AvailableDaysNumber*[t] which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours greater than the *UpperBound*[t]
- $0 \leq$ ***DaysEqualUpperBound*[t]** $\leq$ *AvailableDaysNumber*[t] which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours equal to the *UpperBound*[t]
- $0 \leq$ ***DaysLessLowerBound*[t]** $\leq$ *AvailableDaysNumber*[t], which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours lower than the *LowerBound*[t]
- $0 \leq$ ***DaysLessEqualLowerBound*[t]** $\leq$ *AvailableDaysNumber*[t], which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours lower or equal the *LowerBound*[t]
- $0 \leq$ ***DaysEqualLowerBound*[t]** $\leq$ *AvailableDaysNumber*[t],, which for every teacher t, represent the number of days during which teacher t is assigned a number of teaching hours equal to the *LowerBound*[t]

Equations (15), (18), and (21), listed above, essentially represent conditional constraints, which must be turned into linear constraints. This task may be accomplished using the *Big M* method.

Before presenting the formulation of the above-mentioned constraints, it is appropriate to introduce the *Big M* method:

Let us have the case of a variable U, where $MU2 \leq U \leq MU1$, and try to model the constraint:

$$D = \begin{cases} 1, & if \ U>0 \\ 0, & otherwise \end{cases} \quad (23)$$

Then, according to the *Big M* method, the formulation of this conditional constraint is as follows:

$$U \leq MU1 * D \quad (24.a)$$

and

$$U + MU2 \geq (MU2 + 1) * D \quad (24.b)$$

It is obvious that the formulation of Equations (15), (18), and (21) falls under the same case explained above. However, as will become clear in the next sections, for the formulation of constraints (15) and (21) we employ only equations in the form of (24.a) and we exclude these of the form shown in (24.b). At first glance, this seems false, as without equation (24.b) variable D becomes a fuzzy state variable, as it arbitrary takes the value 0 or 1 when $U \leq 0$. Nonetheless, the use of equations in the form of (24.b) in the formulation of relations (15) and (21) is not only redundant, but dramatically increases the execution time using MIP solvers as well. The reason for considering equations of type (24.b) unnecessary is that constraints (15, 18) and (21) are not independent. Specifically, it suffices to include equations (24.a) and (24.b) only in the formulation of constraint (18), while in the case of constraints (15) and (21) the use of equation (24.a) is sufficient. As demonstrated in the tables subsequently presented, the formulation of constraint (18) using both equations (24.a) and (24.b) ensures the correct formulation of constraints (15) and (21) without the use of (24.b). The following table (Table 1) contains the theoretically possible values for variables $D_{11}[t,d]$, $D_1[t,d]$ and $D_2[t,d]$ in the first three columns, respectively, along with the condition that applies in each case. The fourth column demonstrates the resulting equality for each combination.

Next, we give the reasoning of contradiction for row referred to Combination No 2, ie the row 3 of Table 1. At column $D_{11}[t,d]$, the value of $D_{11}[t,d]$ is assumed to be 0. That means that the double sum $\sum_c \sum_p X[t,c,d,p]$ is smaller than or equal to the value of *LowerBound*[t]. This is easily derived by the definition of $D_{11}[t,d]$ (see paragraph 4.2.2., formula (18)). According to the definition of the binary variable $D_{11}[t,d]$, it takes the value of 0 if $\sum_c \sum_p X[t,c,d,p] < UpperBound[t]$, ie if $\sum_c \sum_p X[t,c,d,p] \leq LowerBound[t]$, since $UpperBound[t] = LowerBound[t] + 1$. At the next column $D_1[t,d]$, the value of $D_1[t,d]$ is assumed to be 0. That means that the double sum $\sum_c \sum_p X[t,c,d,p]$ is greater than or equal to the value of *LowerBound*[t]. This is derived by the definition of the binary variable $D_1[t,d]$ (see paragraph 4.2.2., formula (15)). Now, it is yielded that $\sum_c \sum_p X[t,c,d,p]$ must be equal to the value of *LowerBound*[t]. On the other hand, at column $D_2[t,d]$, the value of $D_2[t,d]$ is assumed to be equal to 1. By the definition of the binary variable $D_2[t,d]$ (see paragraph 4.2.2., formula (21)) it is true that $\sum_c \sum_p X[t,c,d,p]$ is greater than the value of *UpperBound*[t]. Since *UpperBound*[t] is greater than *LowerBound*[t] the contradiction is obviously proven. For the rest cases of rows (ie combinations

Table 1. Value combinations of $D_{11}[t,d]$, $D_1[t,d]$, and $D_2[t,d]$ and resulting $I = \sum_c \sum_p X[t,c,d,p]$

Comb.No	Value of $D_{11}[t,d]$ and condition for I	Value of $D_1[t,d]$ and condition for I	Value of $D_2[t,d]$ and condition for I	Resulting value of $I = \sum_c \sum_p X[t,c,d,p]$
1	0 $I \le LowerBound[t]$	0 $I \ge LowerBound[t]$	0 $I \le UpperBound[t]$	$I = LowerBound$
2	0 $I \le LowerBound[t]$	0 $I \ge LowerBound[t]$	1 $I > UpperBound[t]$	Contradiction $\begin{pmatrix} I = LowerBound[t] \\ I > UpperBound[t] \end{pmatrix}$
3	0 $I \le LowerBound[t]$	1 $I < LowerBound[t]$	0 $I \le UpperBound[t]$	$I < LowerBound$
4	0 $I \le LowerBound[t]$	1 $I < LowerBound[t]$	1 $I > UpperBound[t]$	Contradiction $\begin{pmatrix} I < LowerBound[t] \\ I > UpperBound[t] \end{pmatrix}$
5	1 $I > LowerBound[t]$	0 $I \ge LowerBound[t]$	0 $I \le UpperBound[t]$	$I = UpperBound$
6	1 $I > LowerBound[t]$	0 $I \ge LowerBound[t]$	1 $I > UpperBound[t]$	$I > UpperBound$
7	1 $I > LowerBound[t]$	1 $I < LowerBound[t]$	0 $I \le UpperBound[t]$	Contradiction $\begin{pmatrix} I>LowerBound[t] \\ I<LowerBound[t] \end{pmatrix}$
8	1 $I > LowerBound[t]$	1 $I < LowerBound[t]$	1 $I > UpperBound[t]$	Contradiction $\begin{pmatrix} I>LowerBound[t] \\ I<LowerBound[t] \end{pmatrix}$

Table 2. Feasible value combinations of $D_{11}[t,d]$, $D_1[t,d]$, and $D_2.[t,d]$

Combination No	$I = \sum_c \sum_p X[t,c,d,p]$	$D_{11}[t,d]$	$D_1[t,d]$	$D_2[t,d]$
1	$I = LowerBound[t]$	0	0	0
3	$I < LowerBound[t]$	0	1	0
5	$I = UpperBound[t]$	1	0	0
6	$I > UpperBound[t]$	1	0	1

of $D_{11}[t,d]$, $D_1[t,d]$ and $D_2[t,d]$ values) where it is stated that there is a contradiction, it is proven in a similar way like the one presented for the third row.

As results from Table 1, out of a total of eight combinations only 1, 3, 5, and 6 are feasible. Feasible combinations 1, 3, 5, and 6 are grouped in Table 2, given below:

> Observing Table 2, it becomes evident that it is sufficient to only correctly define constraint $D_{11}[t,d]$ (using equations (24.a) and (24.b)). This happens because combinations 5 and 6, which do not definitively determine if the value of variable $D_2[t,d]$ is 0 or 1, under the same condition for variables $D_{11}[t,d]$ and $D_1[t,d]$, cannot coexist.

In reality, combination 5 holds when $\sum_c \sum_p X[t,c,d,p] = UpperBound[t]$, while combination 6 is valid when $\sum_c \sum_p X[t,c,d,p] > UpperBound[t]$.

Subsequently, we move on to the formulation of constraints (15, 18) and (21) utilising *Big M* Method. We must note that in the following, relation $t \le 35$ implies that teacher t is not a virtual one, since instance 2 includes 35 ordinary teachers, while the rest are virtual teachers. Moreover, equation $d = d * TeacherUnavDays[t,\ d]$ implies that day d is an availability day for teacher t, since $TeacherUnavDays[t,\ d] = 1$ if and only if teacher t is available on day d and $TeacherUnavDays[t,\ d] = 0$ if not.

Conditional constraint (15):

$$MU1 * D_1[t,d] \ge LowerBound[t] - \sum_c \sum_p X[t,c,d,p]$$
$$where\ t \le\ 35,\ d = d * TeacherUnavDays[t,\ d]$$

Conditional constraint (18):

$$\sum_c \sum_p X[t,c,d,p] - LowerBound[t] \le MU2 * D_{11}[t,d]$$
$$where\ t \le 35,\ d = d * TeacherUnavDays[t,d]$$

$$\sum_c \sum_p X[t,c,d,p] - LowerBound[t] + MU1 \ge (MU1 + 1) * D_{11}[t,d]$$
$$where\ t \le 35,\ d = d * TeacherUnavDays[t,d]$$

Conditional constraint (21):

$$MK1 * D_2[t,d] \ge \sum_c \sum_p X[t,c,d,p] - UpperBound[t]$$
$$where\ t \le\ 35,\ d = d * TeacherUnavDays[t,\ d]$$

Moreover, we consider the following additional constraints derived from the previously defined corresponding variables:

1. $DaysGreaterEqualUpperBound[t] = \sum_d D_{11}[t,d]$ $where\ t \le 35,\ d = d * TeacherUnavDays[t,d]$
2. $DaysGreaterUpperBound[t] = \sum_d D_2[t,d]$ $where\ t \le 35,\ d = d * TeacherUnavDays[t,d]$
3. $DaysEqualUpperBound[t] = DaysGreaterEqualUpperBound[t] - DaysGreaterUpperBound[t]$ $where\ t \le 35$
4. $DaysLessLowerBound[t] = \sum_d D_1[t,d]$ $where\ t \le 35,\ d = d * TeacherUnavDays[t,d]$

5. $DaysLessEqualLowerBound[t] = AvailableDays\ Number[t] - DaysGreaterEqualUpperBound[t]$ $where\ t \leq 35$
6. $DaysEqualLowerBound[t] = DaysLessEqual\ LowerBound[t] - DaysLessLowerBound[t] where\ t \leq 35$
7. $DaysEqualLow[t] - LowBoundTimes[t] \leq (Avail\ Days[t]) * D_3[t] where\ t \leq 35$
8. $DaysEqualUpper[t] - UpperBoundTimes[t] \leq (AvailDays[t]) * D_4[t] where\ t \leq 35$

4.2.4. S3 MIP formulation

In order to formulate the **S3** soft constraint, we define the following variable:

$$\Theta = \sum_p X[t,c,d,p] - TeacherClassSubjects[t,c] \quad \forall t \in T, c \in C, d \in D \tag{25}$$

The values for parameters $TeacherClass\ Subjects[t,c]$ are given in the input file.

In addition, for each $t \in T$, $c \in C$ and $d \in D$ we define the binary variable:

$$G[t,c,d] = \begin{cases} 1, & if\ \Theta > 0 \\ 0, & otherwise \end{cases} \tag{26}$$

Obviously, it holds that: $-Z \leq \Theta \leq N$, where, for example, in the case of file 2 it is $Z = 4$ and $N = 6$, as it is easily derived.

In addition, for each $c \in C$ and $d \in D$ we define the binary variable:

$$GG[c,d] = \begin{cases} 1, & if\ at\ least\ one\ of\ G[t,c,d]\ equals\ 1 \\ 0, & otherwise \end{cases} \tag{27}$$

The formulation for the conditional constraint (26), using the *Big M* method, is consisted of inequalities (28) and (29) that follow:

$$\Theta \leq N * G[t,c,d] \tag{28}$$

$$\Theta + Z \geq (Z+1) * G[t,c,d] \tag{29}$$

On the other hand, the formulation of conditional constraint (27) is modelled by inequality (30) that follows:

$$GG[c,d] \geq \frac{\sum_t G[t,c,d]}{TeachersNo} \quad \forall c \in C, d \in D \tag{30}$$

4.3 Objective function for the model

Following the soft constraint formulation presented above, the form of the objective function, which we aim to minimise is as follows:

$$objF = constraint_{S_1} + constraint_{S_2} + constraint_{S_3} \tag{31}$$

where

$$constraint_{S_1} = \sum_t \sum_d \sum_{m,\ n \in U} (n - m - 1) Z[t,d,m,n]$$

$$constraint_{S_2} = \sum_t \sum_d D_1[t,d] + \sum_t \sum_d D_2[t,d] + \sum_t D_3[t] + \sum_t D_4[t]$$

$$constraint_{S_3} = \sum_c \sum_d GG[c,d]$$

4.4. A new model decomposition approach

As previously mentioned, we have utilised two methods to deal with the problem. The first one leads to a so called "monolithic" model, ie a model that consists of all hard and soft constraints. The second one is based on dividing the problem to smaller sub-problems that are easier to solve and is described next. It should be mentioned that by solving the monolithic model we got solutions of very poor quality even if we allowed unreasonably large execution times. On the other hand, the second decomposition method yields very satisfactory results in negligible execution times compared to the allowed execution times of the monolithic model. An idea of the problem difficulty is given by observing Table 3(a) where several parameters of the monolithic model are presented.

The second decomposition method is as follows: unlike the first monolithic method, where all hard and soft constraints are considered in the same model, the second decomposition method considers six sub-models of the main problem. Specifically, a model that includes all hard constraints and only soft constraints S2 and S3 is solved first. Then, parts of this solution corresponding to each day of the week are fed as input to 5 other models, each one for each day respectively. Each of these models includes all hard constraints and soft constraint S1, excluding soft constraints S2 and S3. It should be noted that the solution process of each one of the 5 models can by no means alter the possible violation costs of soft constraints S2 and S3 given by the solution process of the model that includes them. This is possible due to the relative independence of soft constraint S1 soft constraints from S2 and S3. After solving the six sub-models for each instance, we merely aggregate the violation costs of soft constraints S2 and S3 and the five sub-costs of soft constraint S1's violation. Thus, we have the total soft constraint violation cost of the whole week.

Table 3(a). Difficulty parameters of the monolithic model.

Instance	Constraints number	Integer variables number	Binary variables number	Non-zeroes
1	12698	11177	10410	111004
2	14069	12446	11603	123715
3	6395	5237	4839	52791
4	7389	6098	5635	64600
5	2062	2450	2230	18221
7	14838	14305	13311	148059
8	3941	3173	2882	32204
9	5538	4601	4238	48924
10	5983	5086	4634	30090
11	9483	8443	7443	91831

Table 3(b). New lower values and proven optima (2nd decomposition method).

Instance	PSO Global[a]	CSO[a]	GUROBI	CPLEX	Optimality proven?
1	11	11	8[b]	8[b]	No
2	18	18	16	14[b]	No
3	5[c]	5[c]	5[c]	5[c]	Yes (proof by contradiction method)
4	5[b]	5[b]	5[b]	6	No
5	0[c]	0[c]	0[c]	0[c]	Yes (directly)
7	23	24	12	11[b]	No
8	11[c]	11[c]	11[c]	11[c]	Yes (directly)
9	2[b]	2[b]	2[b]	2[b]	No
10	10[c]	10[c]	10[c]	10[c]	Yes (proof by contradiction method)
11	19	18	17[c]	18	Yes (proof by contradiction method)

[a]Skoullis et al. (2017).
[b]Lower values.
[c]Optimal values.

5. Comparative results

We present the best published results for all 10 instances in Table 3(b). These results are produced by two different soft computing algorithms, namely Particle Swarm Optimization (PSO) and Cat Swarm Optimization (CSO). In addition, Table 3(b) presents the new lower values achieved using solvers Gurobi and CPLEX. These values are provided by following the 2nd decomposition method. Please note that even if the six models of the 2nd decomposition method are solved to optimality, the aggregated solution is not guaranteed to be an optimal one. Nevertheless, the 2nd method provides very satisfactory results compared to the results of the 1st method and to those of soft computing algorithms, ie PSO and CSO. As a matter of fact, the 2nd method achieves new lower primal bounds while, in some cases, provides us with optimal solutions. The optimality of these solutions is proven via the contradiction method which is presented next and it is applied to instances 3, 10, and 11. This contradiction-based method for proof of optimality is outlined below for file 3, while the proof process is identical for files 10 and 11.

First, in the case of file 3, model M, the objective function of which only contains soft constraints **S2** (teacher dispersion) and **S3** (class dispersion) produces only the values of 2 and 3 for the cost of **S2** and **S3**, respectively (**S2_cost** = 2 and **S3_cost** = 3). Also, the five models, each of which corresponds to a different weekday and only include constraint **S1** (teacher idle times) in their objective function, produce the value of 0 as the cost for **S1** for each day ($S1\_cost = 0$). So, the total cost is equal to 5. Furthermore, we assume that the value of 5 is not optimal and that an integrated model, which would include all three soft constraints in its objective function, might produce a cost value lower than 5. Since the solution of model ***M*** is optimal, the only case where the total cost could be reduced to a value lower than 5, is if the combined cost of **S2** and **S3** was the same or increased by one unit and possibly the cost of **S1** were reduced for the 5 days. This is not possible, though, as the cost of constraint **S1** is already equal to 0. Consequently, there cannot be any solution featuring a cost value lower than 5, which proves the optimality of the current solution.

Table 4(a), which is given below, presents the comparative results of the two state-of-the art heuristics, ie PSO and CSO. We also show the execution time for the best results of these two soft algorithms. As it is mentioned above the global version of PSO algorithm along with CSO have, to this day, achieved the best results for these 10 instances, compared to the published metaheuristic methods in the respective literature (Beligiannis et al., 2008, 2009; Katsaragakis et al., 2015; Raghavjee & Pillay, 2013, 2015; Tassopoulos & Beligiannis, 2012b;

Table 4(a). Heuristics' results.

	PSO global		CSO	
Instance	Result	Time	Result	Time
1	11	32 m 39 s	11	25 m 16 s
2	18	46 m 57 s	18	32 m 30 s
3	5	3 m 30 s	5	2 m 51 s
4	5	13 m 04 s	5	7 m 35 s
5	0	1 m 55 s	0	2 m 48 s
7	23	57 m 36 s	24	42 m 13 s
8	11	1 m 47 s	11	1 m 09 s
9	2	4 m 21 s	2	2 m 51 s
10	10	3 m 42 s	10	2 m 20 s
11	19	7 m 07 s	18	4 m 52 s

Table 4(b). 1st (Monolithic) method's results.

Gurobi & CPLEX				
Instance	Dual bound	Result	Primala bound	Time
1	3	23	8	4 hours
2	4	53	14	4 hours
3	5	7	5	4 hours
4	0	15	5	4 hours
5	0	0	0	1 s
7	0	279	11	4 hours
8	11	11	11	1 m 30 s
9	0	2	2	27 s
10	10	10	10	20 m
11	17	26	17	4 hours

[a]Provided by 2nd decomposition method.

Table 4(c). 2nd Decomposition method's results.

	Gurobi		CPLEX	
Instance	Result	Time	Result	Time
1	8	1 m 31 s	8	1 m 40 s
2	16	1 m 33 s	14	1 m 28 s
3	5	2 s	5	2 s
4	5	52 s	6	7 s
5	0	< 1 s	0	< 1 s
7	12	6 m 33 s	11	4 m 11 s
8	11	6 s	11	8 s
9	2	3 s	2	5 s
10	10	2 s	10	5 s
11	17	6 m 44 s	18	3 m 20 s

Zhang et al., 2010). Tables 4(b) and 4(c), which are also given below, present the results of solving the monolithic model via the 1st method and the results of the 2nd decomposition method respectively. All the experiments were carried out on a Windows 10 64 bit, Intel i7-5500U, CPU 2.40GHz, 8GB RAM PC.

Comparing results presented in Tables 4(b) and 4(c) we conclude that the results produced by solving the problem via a monolithic model are much worse than those produced by the 2nd decomposition method. Although in Table 4(b) we present the best results produced in at most 4 hours by solvers Gurobi and CPLEX it is true that even when we allowed much more time the results were still worse than those of the 2nd decomposition method. In addition, observing Table 4(c) we verify the well-known fact that the two commercial solvers, namely Gurobi and CPLEX, have almost the same performance.

On the other hand, comparing PSO, global version, and CSO with CPLEX and Gurobi when the 1st method of the monolithic model is considered, we conclude that the two soft computing algorithms perform much better in almost all cases, as they produce better results in shorter time. At the contrary, when the 2nd decomposition method using CPLEX and Gurobi is the case, then the two soft computing algorithms achieve the same result for instances 3, 4, 5, 8, 9, and 10 needing however longer time. As far as the rest instances are concerned, then CPLEX and Gurobi perform generally better than the two soft computing algorithms when the 2nd decomposition method is used.

6. Conclusions and future work

As it is proven, the school timetabling problem belongs to the NP- Complete class of problems. Hence, it is unlikely for this problem to be solved to optimality by a deterministic algorithm in polynomial time. Our conclusion is that this statement is probably correct. All our efforts to solve the problem at hand while all hard and soft constraints are included in the same model via the two MIP solvers, namely Gurobi and CPLEX, were unsuccessful, ie they gave substandard quality results, even if we allow unreasonable large execution times. Nevertheless, we succeeded to get good quality solutions and even optimal ones by examining the structure of the soft constraints and the relation among them. The success of our approach lies in the existent independence of the soft constraint concerning the teachers' idle times to the other two soft constraints. This independence motivated us to utilise a two-phase approach in which we first treated the two soft constraints and then we used sub models, each one corresponding to a day of the week, while taking into account only the teachers' idle time soft constraint. Lach and Lübbecke (2012) state: "Actually, we believe that optimal or high-quality solutions for university course timetabling, also with the enormous progress made in IP solver technologies, will not be computed with off-the-shelf solvers, but will call for tailored algorithms. This requires to further investigate the combinatorial structure hidden in the problem, eg, in the soft constraints, and we are encouraged by our good results to pursue such studies in the future". We absolutely agree with this statement. Our presented work indicates that the latter statement is true in the case of the school timetabling problem, too. We believe that "divide – and – conquer" methods, ie those that decompose a difficult problem to simpler ones, applied to Linear Programming is promising. We further believe that soft computing combined with the strength of modern Linear Programming Solvers has a lot to offer towards achieving good results to a wide class of NP-Complete combinatorial problems. In fact, our future work is

oriented in this direction. We intend to design new algorithms based on Mixed Integer Programming and soft computing techniques in order to deal with well-known timetabling problems.

Disclosure statement

No potential conflict of interest was reported by the authors.

Notes

1. MathProg | http://lpsolve.sourceforge.net/5.5/MathProg.htm
2. AMPL | http://ampl.com/
3. Gurobi Optimization | http://www.gurobi.com/
4. IBM CPLEX Optimizer | https://www-01.ibm.com/software/commerce/optimization/cplex-optimizer/
5. http://www.deapt.upatras.gr/school_timetabling_MIP_modeling
6. http://www.deapt.upatras.gr.school_timetabling_MIP_modeling/ [University of Patras, Greece]

References

Al-Yakoob, S. M., & Sherali, H. D. (2015). Mathematical models and algorithms for a high school timetabling problem. *Computers and Operations Research*, *61*, 56–68. doi:10.1016/j.cor.2015.02.011

Beligiannis, G. N., Moschopoulos, C. N., Kaperonis, G. P., & Likothanassis, S. D. (2008). Applying evolutionary computation to the school timetabling problem: The Greek case. *Computers & Operations Research*, *35*(4), 1265–1280. doi:10.1016/j.cor.2006.08.010

Beligiannis, G. N., Moschopoulos, C., & Likothanassis, S. D. (2009). A genetic algorithm approach to school timetabling. *Journal of the Operational Research Society*, *60*(1), 23–42. doi:10.1057/palgrave.jors.2602525

Birbas, T., Daskalaki, S., & Housos, E. (1997). Timetabling for Greek high schools. *Journal of the Operational Research Society*, *48*(12), 1191–1200. doi:10.1057/palgrave.jors.2600480

Birbas, T., Daskalaki, S., & Housos, E. (2009). School timetabling for quality student and teacher schedules. *Journal of Scheduling*, *12*(2), 177–197. doi:10.1007/s10951-008-0088-2

Bixby, R. E. (2012). A brief history of linear and mixed-integer programming computation. *Documenta Mathematica, Extra, ISMP*, 107–121.

Carter, M. W., & Tovey, C. A. (1992). When is the classroom assignment problem hard? *Operations Research, INFORMS*, *40*, S28–S39. doi:10.1287/opre.40.1.S28

Cooper, T. B., & Kingston, J. H. (1996). The complexity of timetable construction problems. In E. Burke, & P. Ross (Eds.), *Practice and theory of automated timetabling. PATAT 1995. Lecture notes in computer science* (Vol. 1153, pp. 281–295). Berlin, Heidelberg: Springer. doi:10.1007/3-540-61794-9_66

Domrös, J., & Homberger, J. (2012). An evolutionary algorithm for high school timetabling. In Proceedings of the ninth international conference on the practice and theory of automated timetabling, PATAT 2012, Son, Norway, August 2012.

Dorneles, Á. P., de Araújo, O. C., & Buriol, L. S. (2012). The impact of compactness requirements on the resolution of high school timetabling problem. CLAIO/SBPO 2012. Rio de Janeiro, Brazil.

Dorneles, Á. P., de Araújo, O. C., & Buriol, L. S. (2014). A fix-and-optimize heuristic for the high school timetabling problem. *Computers & Operations Research*, *52*(Part A), 29–38. doi:10.1016/j.cor.2014.06.023

Dorneles, Á. P., de Araújo, O. C., & Buriol, L. S. (2017). A column generation approach to high school timetabling modeled as a multicommodity flow problem. *European Journal of Operational Research*, *256*(3), 685–695. doi:10.1016/j.ejor.2016.07.002

Even, S., Itai, A., & Shamir, A. (1975). On the complexity of time table and multi-commodity flow problems. Foundations of Computer Science, 16th Annual Symposium on (pp. 184–193). IEEE.

Fonseca, G. H., Santos, H. G., & Carrano, E. G. (2016). Late acceptance hill-climbing for high school timetabling. *Journal of Scheduling*, *19*(4), 453–465. doi:10.1007/s10951-015-0458-5

Fonseca, G. H. G., Santos, H. G., Toffolo, T. A. M., Brito, S. S., & Souza, M. J. F. (2012). A SA-ILS approach for the high school timetabling problem. In Proceedings of the ninth international conference on the practice and theory of automated timetabling, PATAT 2012, Son, Norway, August 2012.

Katsaragakis, I. V., Tassopoulos, I. X., & Beligiannis, G. N. (2015). A comparative study of modern Heuristics on the school timetabling problem. *Algorithms*, *8*(3), 723–742. doi:10.3390/a8030723

Kheiri, A., Özcan, E., & Parkes, A. J. (2012). HySST: hyper-heuristic search strategies and timetabling. In Proceedings of the ninth international conference on the practice and theory of automated timetabling, PATAT 2012, Son, Norway, August 2012.

Kingston, J. H. (2012). A software library for school timetabling. Retrieved from http://sydney.edu.au/engineering/it/~jeff/khe/

Kristiansen, S., & Stidsen, T. R. (2013). A Comprehensive Study of Educational Timetabling - a Survey. DTU Management Engineering Report No. 8.2013, Technical University of Denmark, Department of Management Engineering.

Kristiansen, S., Sørensen, M., & Stidsen, T. R. (2015). Integer programming for the generalized high school timetabling problem. *Journal of Scheduling*, *18*(4), 377–392. doi:10.1007/s10951-014-0405-x

Lach, G., & Lübbecke, M. E. (2012). Curriculum based course timetabling: new solutions to Udine benchmark instances. *Annals of Operations Research*, *194*(1), 255–272. doi:10.1007/s10479-010-0700-7

Saviniec, L., & Constantino, A. A. (2017). Effective local search algorithms for high school timetabling problems. *Applied Soft Computing*, *60*, 363–373.

Saviniec, L., Santos, M. O., & Costa, A. M. (2018). Parallel local search algorithms for high school timetabling problems. *European Journal of Operational Research*, *265* (1), 81–98.

Papoutsis, K., Valouxis, C., & Housos, E. (2003). A column generation approach for the timetabling problem of Greek high schools. *Journal of the Operational Research Society*, *54*(3), 230–238. doi:10.1057/palgrave.jors.2601495

Pillay, N. (2014). A survey of school timetabling research. *Annals of Operations Research*, *218*(1), 261–293. doi:10.1007/s10479-013-1321-8

Post, G., Gaspero, L. D., Kingston, J. H., McCollum, B., & Schaerf, A. (2013). The third international timetabling

competition. *Annals of Operational Research*, *239*, 69–75. doi: 10.1007/s10479-013-1340-5

Raghavjee, R., & Pillay, N. (2013). A study of genetic algorithms to solve the school timetabling problem. In F. Castro, A. Gelbukh, and M. González (Eds.), *Advances in soft computing and its applications. MICAI 2013. Lecture notes in computer science* (Vol. 8266, pp. 64–80). Berlin, Heidelberg: Springer. doi:10.1007/978-3-642-45111-9_6

Raghavjee, R., & Pillay, N. (2015). A genetic algorithm selection perturbative hyper-heuristic for solving the school timetabling problem. *ORiON*, *31*(1), 39–60. doi: 10.5784/31-1-158

Santos, H. G., Uchoa, E., Ochi, L. S., & Maculan, N. (2012). Strong bounds with cut and column generation for class-teacher timetabling. *Annals of Operations Research*, *194*(1), 399–412. doi:10.1007/s10479-010-0709-y

Schaerf, A. (1999). A survey of automated timetabling. *Artificial Intelligence Review*, *13*(2), 87–127. doi: 10.1023/A:1006576209967

Skoullis, V. I., Tassopoulos, I. X., & Beligiannis, G. N. (2017). Solving the high school timetabling problem using a hybrid cat swarm optimization-based algorithm. *Applied Soft Computing*, *52*, 277–289. doi: 10.1016/j.asoc.2016.10.038

Sørensen, M., Kristiansen, S., & Stidsen, T. R. (2012). International timetabling competition 2011: an adaptive large neighborhood search algorithm. In Proceedings of the ninth international conference on the practice and theory of automated timetabling, PATAT 2012, Son, Norway, August 2012.

Sørensen, M., & Stidsen, T. R. (2013). Comparing Solution Approaches for a Complete Model of High School Timetabling. DTU Management Engineering Report No. 5.2013, Technical University of Denmark, Department of Management Engineering.

Tassopoulos, I. X., & Beligiannis, G. N. (2012a). Using particle swarm optimization to solve effectively the school timetabling problem. *Soft Computing*, *16*(7), 1229–1252. doi:10.1007/s00500-012-0809-5

Tassopoulos, I. X., & Beligiannis, G. N. (2012b). A hybrid particle swarm optimization-based algorithm for high school timetabling problems. *Applied Soft Computing*, *12*(11), 3472–3489. doi:10.1016/j.asoc.2012.05.029

ten Eikelder, H. M., & Willemen, R. J. (2001). Some complexity aspects of secondary school timetabling problems. In E. Burke and W. Erben (Eds.), *Practice and theory of automated timetabling III. PATAT 2000. Lecture Notes in Computer Science* (Vol. 2079, pp. 18–27). Berlin, Heidelberg: Springer. doi:10.1007/3-540-44629-X_2

Valouxis, C., & Housos, E. (2003). Constraint programming approach for school timetabling. *Computers & Operations Research*, *30*(10), 1555–1572. doi:10.1016/S0305-0548(02)00083-7

Willemen, R. J. (2002). School timetable construction: Algorithms and complexity (PhD Thesis). Technische Universiteit Eindhoven, The Netherlands.

Zhang, D., Liu, Y., M'Hallah, R., & Leung, S. C. (2010). A simulated annealing with a new neighborhood structure-based algorithm for high school timetabling problems. *European Journal of Operational Research*, *203*(3), 550–558. doi:10.1016/j.ejor.2009.09.014